Hybrid Learning Schemes for Multimedia Information Retrieval

Wei-Cheng Lai and Edward Y. Chang

Electrical & Computer Engineering, University of California, Santa Barbara, CA 93106

Email: echang@ece.ucsb.edu

Abstract. Traditional database systems assume that precise query concepts can be specified by users (for example, by using query languages). For many search tasks, however, a query concept is hard to articulate, and articulation can be subjective. Most users would find it hard to describe an image or a music query in low-level perceptual features. We believe that one desirable paradigm for search engines is to mine (i.e., to learn) users' query concepts through *active learning*. In this paper, we formulate the query-concept learning problem as finding a binary classifier that separates relevant objects from those that are irrelevant to the query concept. We first present two base learning algorithms that we have developed, and then propose three hybrid algorithms, *pipeline learning*, *transductive learning*, and *co-training*, for improving the base algorithms. Our empirical study shows that even when the feature dimension is very high and target concepts are very specific, the hybrid algorithms can grasp a complex query concept in a small number of user iterations.

1 Introduction

A database system cannot answer a query without knowing the query criteria (i.e., *query concept*). For querying a relational database, a query concept is conveyed via query languages. For querying with an Internet search engine, a concept is conveyed through keywords. For querying a multimedia database, however, a query concept is hard to articulate in query languages, and articulation can be subjective. For instance, in an image search, it is difficult for a user to describe desired images using low-level features such as color, shape and texture. In addition, different users may perceive the same image differently (i.e., subjectively). Even if an image is perceived similarly, users may use different vocabulary (i.e., different combinations of low-level features and keywords) to depict it. In order to make query formulation easy and personalized, it is both necessary (for capturing abstract and subjective concepts) and desirable (for alleviating difficulties from specifying complex query concepts) to build intelligent search engines that can quickly learn users' query concepts.

For learning complex and subjective query concepts with a small amount of training data, we have proposed two *active learning* algorithms—MEGA and SVM_{Active} [1, 2]. In this paper, we present hybrid approaches to improve the effectiveness of these base algorithms. We regard the problem of learning a query concept as learning a binary classifier that separates the data objects relevant to the query criteria from the irrelevant ones. The learning task faces three challenging steps:

1. Initialization. The learner must quickly identify some positive and some negative examples to obtain a fuzzy boundary separating the objects relevant to the query concept from irrelevant ones.
2. Refinement. The learner must then refine the classification boundary with a minimum number of labeled instances (i.e., with minimum rounds of relevance feedback).
3. Ranking. When the binary classifier is learned with sufficient confidence, the learner must quickly return the top-k objects that are relevant to the query concept.

Our proposed hybrid schemes for tackling the above challenges are as follows:

1. *Pipeline training.* We combine the strengths of MEGA (finding initial relevant objects to the concept) and SVM_{Active} (refining the class boundary), to show that using MEGA first and then switching to SVM_{Active} can enable the learner to learn a concept substantially faster and more accurately than the individual algorithms can.
2. *Transductive learning aided by committee co-training.* When the number of relevant objects is significantly less than the irrelevant ones, a substantially larger number of negative-labeled instances are needed to depict the class boundary. We propose a new approach, which combines transductive learning and co-training of a committee of learners (including MEGA and SVM_{Active}), to effectively find more negative-labeled instances to speed up the learning process.
3. *Recursive subspace co-training.* This approach divides the feature space into subspaces and conducts recursive training in these subspaces. The initial training data can in different subspaces infer different negative-labeled instances. The inferred negative-labeled instances can be added to the training data to retrain the binary classifier in all subspaces to infer even more negative-labeled instances. This recursive procedure continues until few negative-labeled instances can be further inferred.

Given the same number of feedback rounds (or the same amount of training data), our empirical study shows that these hybrid schemes can learn a query concept with higher accuracy when compared to the base algorithms.

The rest of the paper is organized into seven sections. Section 2 describes how the base algorithms work. Section 3 presents our hybrid learning strategies. Sections 4 and 5 report experimental results. Finally, we offer our concluding remarks in Section 6.

2 Base Active Learning Algorithms

Here, we describe MEGA and SVM$_{Active}$, two online learning algorithms that we have recently developed. MEGA is an inductive-based active learning algorithm; SVM$_{Active}$ is a margin-based active learning algorithm. For detailed algorithms and extensive experimental results, please refer to [3, 2].

2.1 MEGA

The first challenge of query concept learning is to find some relevant objects so that the concept boundary can be fuzzily identified. Finding a relevant object can be difficult if only a small fraction of the dataset satisfies the target concept. For instance, suppose the desired objects in a one-million-image dataset is 100 (0.1%). If we randomly select 20 objects per round for users to identify relevant objects, the probability of finding a positive sample after five rounds of random sampling is just 10%—clearly not acceptable.

We can improve the odds with an intelligent sampling method MEGA (The Maximizing Expected Generalization Algorithm), which finds relevant samples quickly, to initialize query-concept learning. MEGA models query concepts in k-CNF [4], which can formulate virtually all practical query concepts[1]. MEGA uses k-DNF to bound the sampling space from which to select the most informative samples for soliciting user feedback.

Definition 1: k-**CNF**: For constant k, the representation class k-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each c_i is a disjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. (A Boolean variable represents an image feature.) No prior bound is placed on θ.

Definition 2: k-**DNF**: For constant k, the representation class k-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each d_i is a conjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

MEGA initializes the query concept-space (QCS) as a k-CNF and the candidate concept-space (CCS) as a k-DNF. The QCS starts as the most specific concept and the CCS as the most general concept. The target concept is more general than the initial QCS and more specific than the initial CCS. The learner learns the QCS, while at the same time refining the CCS to delimit the boundary of the sampling space.

[1] k-CNF is more expressive than k-term DNF, and it has both polynomial sample complexity and time complexity [5, 6].

To learn a target concept, the well-known Valiant's algorithm [7] uses positive-labeled instances to refine QCS and negative-labeled instances to refine CCS. The key difference between Valiant's algorithm and MEGA lies in how training samples are selected. Valiant's algorithm employs a random sampling strategy. MEGA makes sure that each sample can be most useful, and hence reduces the number of samples needed to learn a concept by employing two strategies.

1. Bounding the sample space: Avoid choosing useless unlabeled instances by using the CCS and QCS to delimit the sampling boundary.

2. Maximizing the usefulness of a sample: Choose an example that will remove the maximum expected number of disjunctive terms from QCS. In other words, we choose an example that can maximize the expected generalization of the concept. Even if the example is labeled negative by the user, it can be useful to remove conjunctive terms in the CCS.

It may appear that if we pick an example that has more dissimilar disjunctions (compared to the QCS), we would have a better chance of eliminating more disjunctive terms. This is, however, not true. An example must be labeled by the user as positive to be useful for refining QCS. Unfortunately, an example is less likely to be labeled positive when it has more disjunctions that are dissimilar to the target concept. Therefore, there is a tradeoff between choosing an example that has more contradictory terms and choosing one that is more likely to be labeled positive.

Let Ψ denote the number of disjunctions remaining in the concept. The number of disjunctions that can be eliminated in the current round of sampling, denoted as ψ, is between zero and Ψ. We can write the probability of eliminating ψ terms as $P_e(\psi)$. $P_e(\psi)$ is a monotonically decreasing function of ψ. We attempt to find the ψ that can eliminate the maximum expected number of disjunctive terms, given an example. Our objective function can be written as

$$\psi^* = arg_\psi \max E(\psi) = \psi \times P_e(\psi). \quad (1)$$

To solve ψ^*, we must know $P_e(\psi)$, which can be estimated by the two methods below: probabilistic estimation and empirical estimation. (Please refer to [3] for detailed discussion.)

2.2 SVM$_{Active}$

Once some relevant and some irrelevant samples are marked, we can employ SVM$_{Active}$ to refine the class boundary. SVMs have strong theoretical foundations and excellent empirical successes. They have been applied to tasks such as handwritten digit recognition [8], object recognition [9], and text classification [10]. We added the active learning component to SVMs for selecting the most informative samples to query a user and to quickly refine a boundary that separates data objects that satisfy the user's query concept from the rest of the dataset.

For the purpose of query-concept learning, we consider SVMs in the binary classification setting. We are given training data $\{\mathbf{x}_1 \ldots \mathbf{x}_n\}$ that are vectors in some space $\mathsf{X} \subseteq \mathbb{R}^d$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin. All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space X to a higher dimensional feature space F via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i K(\mathbf{x}_i, \mathbf{x})$. When $f(\mathbf{x}) \geq 0$ we classify $\mathbf{x}$ as $+1$, otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [11] we can write: $K(\mathbf{u}, \mathbf{v}) = \Phi(\mathbf{u}) \cdot \Phi(\mathbf{v})$ where $\Phi : \mathsf{X} \rightarrow \mathsf{F}$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \Phi(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{i=1}^{n} \alpha_i \Phi(\mathbf{x}_i). \qquad (2)$$

Thus, by using K we are implicitly projecting the training data into a different (often higher dimensional) feature space F. The SVM then computes the α_is that correspond to the maximal margin hyperplane in F. By choosing different kernel functions, we can implicitly project the training data from X into spaces F for which hyperplanes in F correspond to more complex decision boundaries in the original space X.

Intuitively, SVM_{Active} works by combining the following three ideas:

1. SVM_{Active} regards the task of learning a target concept as one of learning an SVM binary classifier. An SVM captures the query concept by separating the relevant images from the irrelevant images with a hyperplane in a projected space, usually a very high-dimensional one. The projected points on one side of the hyperplane are considered relevant to the query concept and the rest irrelevant.

2. SVM_{Active} learns the classifier quickly via active learning. The active part of SVM_{Active} selects the most informative instances with which to train the SVM classifier. This step ensures fast convergence to the query concept in a small number of feedback rounds.

3. Once the classifier is trained, SVM_{Active} returns the top-k most relevant images. These are the k images farthest from the hyperplane on the query concept side.

3 Hybrid Algorithms

Although MEGA and SVM_{Active} are effective active learning algorithms, we believe combining their strengths can result in even better learning algorithms. In this section we propose three such hybrid algorithms:

- Pipeline learning (Section 3.1), and
- Transductive learning aided by committee co-training (Section 3.2).
- Subspace co-training (Section 3.3).

3.1 Pipeline Learning

As mentioned in Sections 1 and 2 learning a query concept faces three challenging steps:

1. Initialization. The learner must quickly identify some positive and some negative examples to obtain a fuzzy boundary separating the objects relevant to the query concept and the others.
2. Refinement. The learner must then refine the classification boundary with minimum number of labeled instances (i.e., with minimum rounds of relevance feedback with impatient users).
3. Ranking. When the binary classifier has been learned with sufficient confidence, the learner must quickly return the top-k objects that are relevant to the query concept.

As depicted in Section 2.2, the SVM_{Active} scheme needs at least one positive and one negative example to start. MEGA is not restricted by this seeding requirement, and it is able to find relevant examples quickly by refining the sampling boundary. It is therefore logical to employ MEGA to perform the *initialization* task. Once some relevant images are found, the refinement step can be executed by either MEGA or SVM_{Active}.

Thus, we can have three execution alternatives.

1. MEGA only. Use MEGA all the way to learn a concept.
2. SVM_{Active} only. Use random sampling to find the first relevant example(s) and then use SVM_{Active} to learn a concept.
3. Pipeline learning. Use MEGA to find initial relevant objects, and then switch to SVM_{Active} for refining the binary classifier and ranking returned objects.

3.2 Transductive Learning

The major challenge that our learning task faces is the scarcity of training data. As counter-intuitive as this may sound, the scarcity of negative examples is more problematic than the scarcity of positive examples for our learning algorithms. This is because describing a concept such as "tigers" may be challenging, but describing a negated concept (in this case the "non-tiger" concept) can require potentially infinite information. (The number of images needed to adequately portrait the tiger concept is finite, but the number of non-tiger images is infinite.) Negative examples are significantly easier to come by, but at the same time, the number of negative samples needed to depict a negated concept can be very large. Thus, we need a substantial number of negative examples to accurately characterize the class boundary.

Let us revisit the SVM$_{Active}$ algorithm. At each round of sampling, it selects objects that are in the margin and also close to the dividing hyperplane. If the margin is wide (because of the lack of support at the negative-example side), the probability is low that a positive example can be found in the wide margin. Our goal is to find enough positive examples to depict the target concept. However, the difficulty in gathering sufficient negative examples slows down the progress for finding positive examples. The learning process is analgous to mining gold in a stream. The most productive way to harvest gold (i.e., relevant instances) is to quickly filter out water, mud and sand (i.e., negative-labeled instances).

Transduction For solving the problem of insufficient training data, transductive learning has been proposed to work with different learning algorithms for various applications [12, 13]. The basic idea of transduction is to leverage the unlabeled data near the labeled data. Suppose the nearest neighbors of negative examples are always negative. We can then use the neighboring examples to substantially increase the negative-labeled pool. However, transduction must be performed with care, since the mis-labeling of data caused by wrong "guesses" can degrade the learning result.

The performance of transduction is, however, consistently inconsistent. Its performance could not only be application-dependent, but also dataset-dependent. For image retrieval, traditional transduction may not be suitable for the following two reasons:

1. Nearest-neighbor sampling. Most image query concepts are not convex, and do not reside continuously in the feature space. For instance, flowers of different colors tend to spread out in the color-feature dimension. Thus, to describe the non-flower concept, we need to explore the feature space more aggressively. Suppose a picture of lake (one of many non-flower instances) is labeled negative. Using the neighboring pictures of this lake picture, which tend to be lake-like or something visually similar, may not be very productive in improving the description of the negated concept.
2. Noise factor. Again, mislabeled data can reduce the prediction accuracy of the classifier.

Co-training We believe that transduction can be helpful, but it must be applied more intelligently. For this goal, we can refine our learning objective slightly: Rather than learning a binary classifier, we learn a K class classifier, which separates the desired class from the other $K-1$ classes. Our objective is thus to find representative examples in the $K-1$ negative classes to describe the negated concept. Note that traditional nearest-neighbor transductive learning tends to produce negative examples in the already known negative classes, and hence the benefit can be of limited usefulness.

To find negative examples in the other $K-1$ classes, the transduction must be more explorative. We propose a committee co-training scheme by employing MEGA and SVM$_{Active}$ as the committee members to aid this explorative process. The presence of two distinct views of each example suggests strategies in which two learning algorithms can be trained separately in each view, and then each algorithm's predictions for new unlabeled instances are used to enlarge the training set of the other. We propose the following committee co-training steps:

1. Use SVM$_{Active}$ to select samples and present them to the user for feedback.
2. Use MEGA and the negative-labeled instances to refine CCS (the k-DNF that bounds the sampling space of MEGA).
3. As discussed in Section 2.1, the concepts that are more general than CCS do not satisfy the target concept. All training data outside of the MEGA's sampling boundary can thus be labeled negative. They can be used to feed SVM$_{Active}$ for refining the hyperplane.
4. Go back to Step 1 until no more negative-labeled instance can be inferred.

3.3 Recursive subspace Co-training

The problem of using a large unlabeled sample pool to boost performance of a learning algorithm is considered under the framework of co-training [14]. A broad definition of co-training is to provide each instance multiple distinct views. We have just shown that distinct views can be provided by different learning algorithms. Here, we propose another co-training method, which provides each training instance distinct views via subspace learning. This method recursively conducts subspace co-training at each feedback iteration in the following steps:

1. Divide the feature space into G subspaces.
2. Conduct parallel training in these G subspaces using labeled training dataset L.
3. Use the G resulting learners to label the unlabeled pool and yield a new set of labeled instances L'.
4. $L \leftarrow L \cup L'$
5. Go back to Step 1 until no more negative-labeled instance can be inferred (i.e., until $L' = \emptyset$).

4 Experiments

We implemented both MEGA and SVM$_{Active}$ in C, $C++$, and tested using an Intel Pentium III^{TM} workstation running Linux. We have implemented an industrial strength prototype [15, 16] with all features discussed in this paper. We tested our prototype with the intent of answering three central questions.

1. Does using MEGA to find the first relevant object(s) and then switching to SVM$_{Active}$ a more effective algorithm than using the two learning algorithms individually?

2. Does the percentage of relevant data affect learning performance? (If we reduce the matching data from 5% of the dataset to 1%, does it take more iterations to learn query concepts?)
3. Do additional heuristics such as transduction and co-training help accelerate concept convergence?

4.1 Data

For our empirical evaluation of our learning methods we used a real-world image dataset: a twenty-category image dataset where each category consisted of 200 to 300 images. Images for this dataset were collected from Corel Image CDs and the Internet.

- *Twenty-category* set. The set consists of images of *architecture*, *bears*, *cities*, *clouds*, *couples*, *flowers*, *insects*, *ladies*, *landscape*, *objectionable images*, *planets*, *tigers*, *tools*, *waves*, *elephants*, *fabrics*, *fireworks*, *food*, and *textures*.

To enable an objective measure of performance, we assumed that a query concept was an image category, or a superset or a subset of a category. For instance, a query can be "wild animals" including "tigers", "elephants", and "bears." A query can be "white bears" (a subset of "bears"), or "purple flowers" (a subset of "flowers"). The learners do not have prior knowledge about image categories. Unlike some recently developed systems [17] that contain a semantical layer between image features and queries to assist query refinement, our system does not have an explicit semantical layer. We argue that having a hard-coded semantical layer can make a retrieval system restrictive to some predefined concepts. Rather, dynamically learning the semantics of a query concept is more flexible and hence makes the system more useful. Each image as a 144-dimension vector [2].

4.2 Queries

We separate our queries into two categories: 5% and 1% queries. The matching images for each of the more specific query concepts such as "purple flowers" and "white bears" account for about 1% of the total dataset. More general concepts such as "bears", "flowers" and "architectures" have about 5% matching images in the dataset. For each experiment, we will report results for these two categories of queries separately. In the experiment, we report precisions of the top-10 and top-20 retrievals to measure performance.

5 Results and Discussion

Here we present our analysis of the results, organized with regard to the central questions listed at the start of Section 4.

5.1 MEGA, SVM$_{Active}$, and Pipelining MEGA with SVM$_{Active}$

For top-10 retrieval with 5% matching data (Figure 1(a)), SVM$_{Active}$ clearly outperforms MEGA. The major weakness of SVM$_{Active}$ is in initialization—finding the first few positive samples. Such weakness may not seriously affect this experiment, since there is a high probability of finding one of the 5% positive examples through random sampling. For queries with only 1% matching data (Figure 1(b)), such a weakness becomes more significant, because it substantially degrades SVM$_{Active}$'s performance. Especially, its precision in the first two iterations is very low. Overall, the precision of MEGA and SVM$_{Active}$ is similar for 1% queries.

The hybrid algorithm (pipelining MEGA with SVM$_{Active}$) clearly outperforms both SVM$_{Active}$ and MEGA. The difference in precision is more significant for queries with 1% matching data than for those with 5% matching data. This trend indicates the strength of the hybrid algorithm in handling more specific query concepts and/or larger datasets. Note that the precision of the hybrid algorithm reaches 100% after 6 and 10 iterations, respectively, for these two experiments (Figures 1(a) and (b)).

For top-20 retrieval (Figures 2(a) and (b)), the precision of the hybrid algorithm remains the highest, followed by SVM$_{Active}$ and MEGA. Again, the differences in their performances are more significant for queries with 1% matching data than for those with 5% matching data. For 1% queries, the hybrid algorithm achieves 100% precision after 12 iterations, which indicates the robustness of this new enhancement.

5.2 Transductive Learning

This experiment examines the effect of transductive learning on the image dataset. We divide our investigation into two parts:

- The noise factor of the dataset.
- Transduction on negative examples.

Noise of the Dataset For the transduction heuristic to be effective, the nearest neighbors of a negative-labeled instance should have a high probability of being negative. Similarly, positive-labeled images should be truly positive. We first analyzed the dataset to investigate whether this requirement is met for this dataset. Figure 3 shows the curves of the probability that the nearest neighbors of a labeled instance have the same label that it does. Figure 3(a) shows such an average probability for each negative-labeled instance after the first and third iterations, respectively. In either of these two curves, a point $(x\%, y\%)$ indicates that, on the average, if an instance in the dataset is within $[(x-5)\%, x\%]$ nearest neighbors of a negative-labeled instance, the probability of the instance being negative is $y\%$. What these curves reflect is that an instance closer to

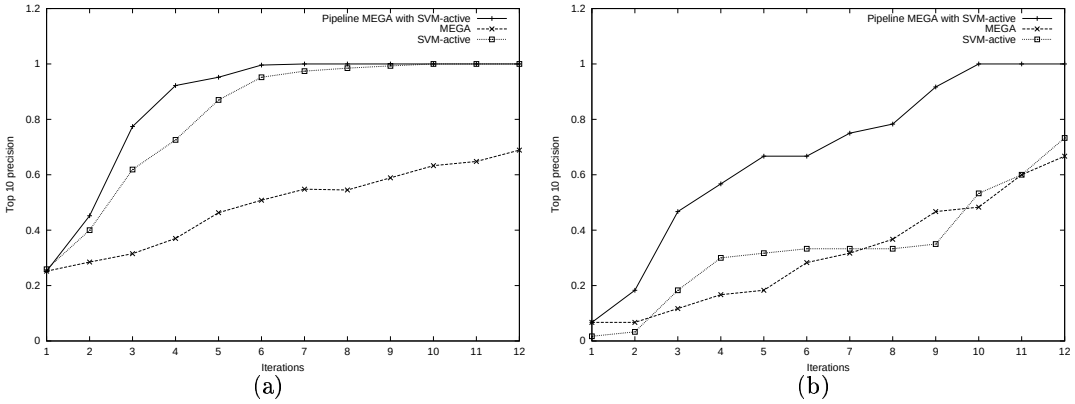


Fig. 1. Precision versus iterations for top-10 retrieval (a) 5% and (b) 1% queries.

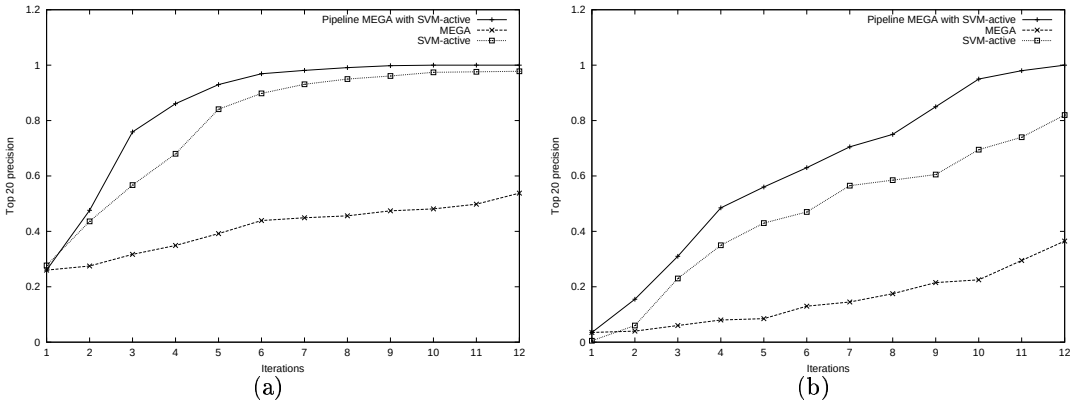


Fig. 2. Precision versus iterations for top-20 retrieval (a) 5% and (b) 1% queries.

a negative-labeled instance is not necessarily more likely to be negative than a more distant instance would be. For example, the closest 5% instances of a negative instance is less likely to be negative than an instances within 5% to 10% of nearest neighbors. The probability of the former is 99.9%, of the latter, 100%. These results indicate that using negative-labeled instances for transduction may introduce noise to and hence may degrade precision.

Similarly, Figure 3(b) shows the average probability of the top 20 nearest neighbors being positive for each positive-labeled instance, after the first and third iterations, respectively. For example, after three iterations, the probability that the nearest neighbor of a positive-labeled instance will be positive is less than 50%. The chance of the second nearest neighbor being positive is less than 15%. This indicates that using positive instances for transduction could likely be counter-productive. Our experimental results indeed show that transduction on positive examples did not help. Due to space limitation, we present just the results of transduction on negative examples next.

Transduction on Negative Examples Even though the majority of the instances are negative for most queries, using the nearest neighbors of the negative-labeled instances as transductive points has a non-zero probability of including some positive instances. To investigate the impact on precision caused by such noise, we compare the results of the following two experiments:

1. Noise-free transduction. We "peek" at the true labels of the nearest neighbors and exclude instances that are positive (but not yet labeled by the user) as negative, transductive points. This experiment will reveal the effectiveness of transduction in an ideal situation—no mislabeled data is included in transductive learning.
2. Normal transduction with noise. This experiment assumes the nearest neighbors of negative-labeled instances to be negative. This scheme has non-zero probability of including positive instances as negative transductive points, as indicated by the results given in Figure 3(a).

Figure 4(a) shows the results for noise-free transduction using the pipeline algorithm as the underlying query concept learner. This experiment uses 1% queries. Each curve shows the precision of top-10 retrieval versus the percentage of the nearest neighbors used as negative transductive points. The three curves show the results after the first, second, and third iterations. Surprisingly, even without any noise, the improvement in precision is only moderate, even if 50% of the dataset images are used as (negative) transductive points.

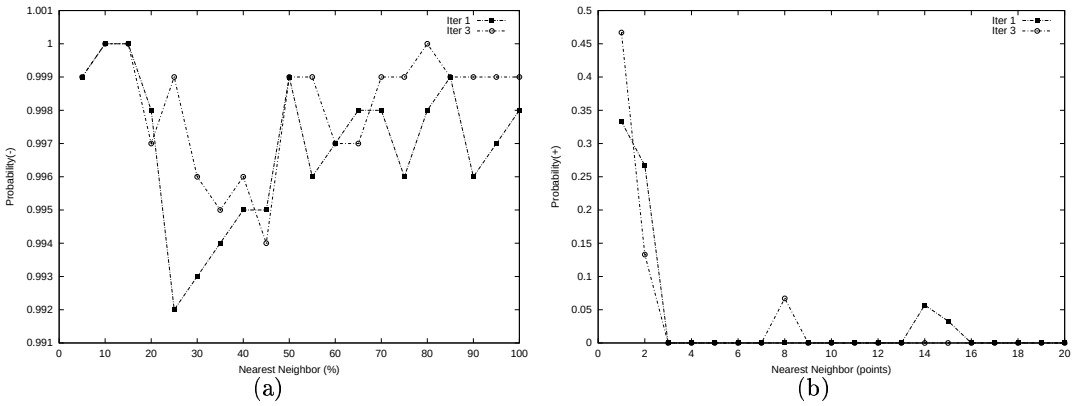


Fig. 3. Probability of the nearest neighbors of a labeled instance having the same label. (a) Probability of being negative versus the relative distance to a negative-labeled instance. (b) Probability of being positive for the top 20 nearest neighbors of a positive-labeled instance.

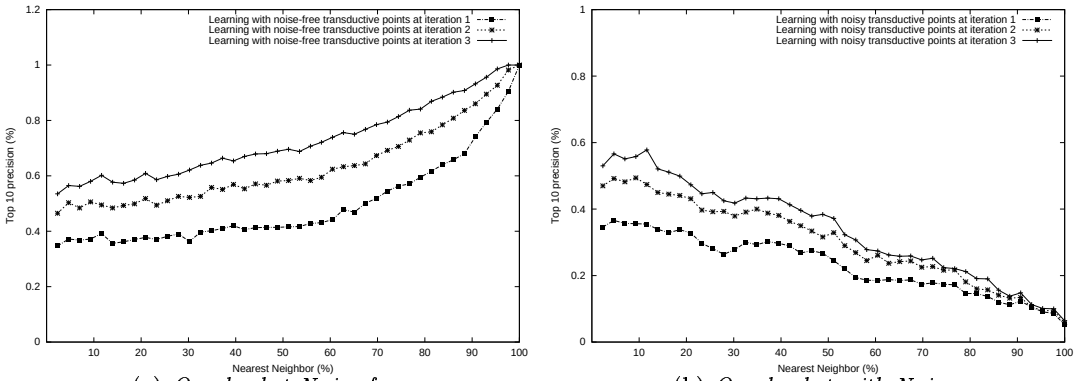


Fig. 4. Precision of top-10 retrieval versus percentage of dataset used as negative transductive points. The transductive points are nearest neighbors of negative-labeled instances.

For normal (and noisy) transduction (Figure 4(b)), transduction does not seem to help. If more than 12% of the dataset is used for transduction, the precision starts to degrade. Note that in this experiment, 99% of the dataset is negative for each query. These results indicate that the noise (i.e. the inclusion of some positive instances as negative transductive points) is the major cause of the ineffectiveness of transduction.

5.3 Committee and Subspace Co-training

We examined the effective of the committee co-training and subspace co-training schemes. For conducting committee co-training, we collected 100 additional negative instances inferred by MEGA (i.e., additional 100 unlabeled points outside CCS) at the end of each user iteration and fed them to SVM_{Active} to refine the class boundary. For conducting subspace co-training, we divide the features into 20 subsets (please refer to [1] for our feature-partition heuristics), and conduct recursive subspace training.

Figures 5(a) and (b) show the curves of precision versus iterations for 1% queries with and without co-training. Co-training indeed improves the precision for both top-10 and top-20 retrievals. On average, the committee co-training scheme achieves 5% higher precision over the pipeline scheme, and the subspace co-training scheme achieves 5−10% higher precision over the committee co-training scheme. For a 1% query, the recursive subspace co-training scheme can reach 70% precision for a top-10 retrieval, and 60% for a top-20 retrieval after the fourth feedback iteration.

5.4 Observations

Our experiments have answered the four questions that we stated in the beginning of Section 4.

1. MEGA and SVM_{Active} can learn complex query concepts in high-dimensional spaces in a small number of user iterations. Compared to some recent schemes proposed by the database community such as Mindreader

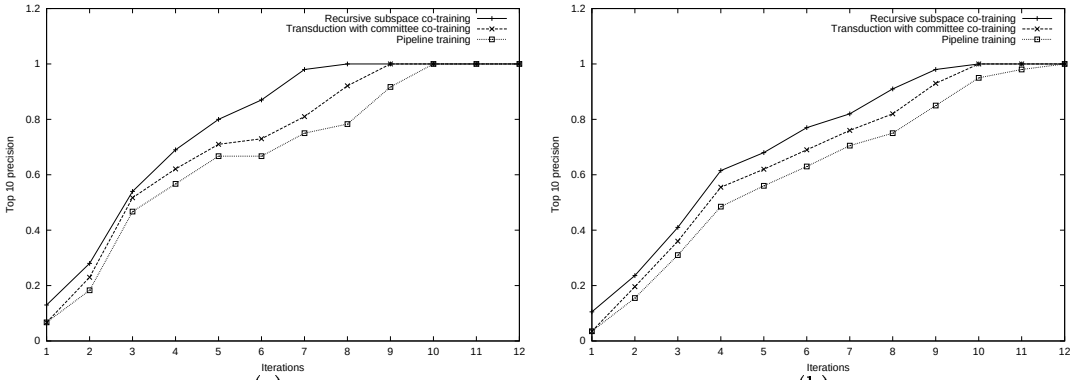


Fig. 5. Precision versus iterations for (a) top-10 retrieval and (b) top-20 retrieval for 1% matching objects with and without co-training.

[18] and Falcon [19], MEGA and SVM_{Active} perform substantially better.

2. The hybrid scheme that uses MEGA to find the first relevant object(s) and then switches to SVM_{Active} works significantly better than using the two learning algorithms individually.
3. When the matching data is scarce, the number of iterations required to learn a concept increases.
4. We show co-training is a good strategy for accelerating concept convergence, whereas nearest-neighbor transductive learning does not help.

6 Conclusion

This paper proposes using hybrid active learning schemes to capture complex and subjective query concepts. We have proposed using MEGA to quickly find objects relevant to the query concept, then switching to SVM_{Active} once relevant objects are found. Our experimental results show that this pipeline approach outperforms MEGA and SVM_{Active} when they are used individually. We have also proposed using transductive learning and co-training to find more useful negative-labeled instances to accelerate the progress of learning. All these hybrid schemes show improved learning performance over the based algorithms.

References

[1] Edward Chang and Beitao Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/~echang/mega.ps*, November 2000.

[2] Simon Tong and Edward Chang. Support vector machine active learning for image retrieval. *Proceedings of ACM International Conference on Multimedia*, pages 107–118, October 2001.

[3] Edward Chang and Beitao Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/~echang/mega.pdf*, November 2000.

[4] Michael Kearns, Ming Li, and Leslie Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.

[5] M.J. Kearns and U.V. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[6] T. Michell. *Machine Learning*. McGraw Hill, 1997.

[7] L. Valiant. A theory of learnable. *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, pages 436–445, 1984.

[8] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[9] C. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In *Proceedings of the International Conference on Computer Vision*, 1998.

[10] T. Joachims. Text categorization with support vector machines. In *Proceedings of the European Conference on Machine Learning*. Springer-Verlag, 1998.

[11] Christopher Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

[12] Thore Graepel, Ralf Herbrich, and Klaus Obermayer. Bayesian transduction. *Advances in Neural Information Systems Processing*, 12, 2000.

[13] T. Joachims. Transductive inference for text classification using support vector machines. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 200–209. Morgan Kaufmann, 1999.

[14] A. Blum and T. Mitchell. *Combining Labeled and Unlabeled Data wih Co-Training*. Proceedings of the Workshop on Computational Learning Theory, 1998.

[15] Edward Chang, Kwang-Ting Cheng, and Lisa Chang. PBIR — perception-based image retrieval. *ACM Sigmod (Demo)*, May 2001.

[16] Edward Chang, Kwang-Ting Cheng, Wei-Cheng Lai, Ching-Tong Wu, Cheng-Wei Chang, and Yi-Leh Wu. PBIR — a system that learns subjective image query concepts. *Proceedings of ACM Multimedia, http://www.mmdb.ece.ucsb.edu/~demo/corelacm/*, pages 611–614, October 2001.

[17] James Wang, Jia Li, and Gio Wiederhold. Simplicity: Semantics-sensitive integrated matching for picture libraries. *ACM Multimedia Conference*, 2000.

[18] Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.

[19] Leejay Wu, Christos Faloutsos, Katia Sycara, and Terry R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The 26th VLDB Conference*, September 2000.